



FUNÇÃO (MODULARIZAÇÃO)

Uma função é um bloco de código **reutilizável** que executa uma tarefa específica. Ela ajuda a **organizar** o programa, **evita repetição** e **facilita a manutenção**.



Aula13- Funções



FUNÇÕES EM JAVASCRIPT

(MODULARIZAÇÃO)



Funções permitem dividir o código em blocos reutilizáveis, organizando melhor o programa e facilitando a manutenção.



O QUE É UMA FUNÇÃO?

É um bloco de código que executa uma tarefa específica.

Você define os passos uma vez e pode usar sempre que precisar.



Analogia: Receita

Uma receita (função) tem ingredientes (parâmetros) e passos (código). Sempre que fizer, o resultado será o esperado!



PARA QUE USAR FUNÇÕES?

- ✓ Reutilização de código
- ✓ Mais organização
- ✓ Facilidade para testar e depurar
- ✓ Redução de erros
- ✓ Manutenção mais simples
- ✓ Programas mais legíveis

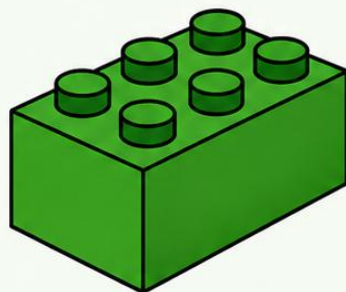


Resumo: Funções transformam tarefas repetitivas em blocos reutilizáveis, tornando o código mais limpo, seguro e eficiente.

ANALOGIA: BLOCOS LEGO

Cada peça (função) tem uma função específica.
Você encaixa essas peças para construir algo maior.

Bloco (Função)



Faz uma tarefa
específica.



Construção (Programa)



Junta várias funções
para resolver o problema.



Função bem feita = peça LEGO de qualidade: encaixa fácil e funciona sempre!



POR QUE USAR FUNÇÕES?



Reutilização
Evita repetição
de código.



Organização
Deixa o código mais
limpo e legível.



Manutenção
Facilita encontrar e
corrigir problemas.



Escalabilidade
Facilita o crescimento
do projeto.



Função = bloco reutilizável
que executa uma tarefa.



Você define uma vez
e usa quantas vezes
precisar.



Melhora a organização
e torna o programa mais
limpo e eficiente.



Facilita manutenção,
testes e evita
duplicação de código.



Programas modulares
são mais fáceis de entender,
expandir e corrigir!



COMO FUNCIONA?

1. Definir



Você cria a função e define o que ela deve fazer.



2. Chamar



Você chama (invoca) a função quando precisa da tarefa.



3. Executar



O código da função é executado com os valores informados.



4. Retornar



A função pode retornar um valor (ou não).



Importante!

- A função só é executada quando é chamada.
- Você pode chamar a mesma função várias vezes com valores diferentes.



SINTAXE BÁSICA

Sem parâmetros

```
function saudacao() {  
    console.log("Olá, mundo!");  
}  
  
saudacao(); // Olá, mundo!
```

Definição da função

Execução (chamada)

Resultado

Com parâmetros

```
function saudacao(nome) {  
    console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana");    // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```

Parâmetro
(nome)



Usado dentro
da função



Dica:

O nome da variável passada na chamada (ex.: nome) pode ser diferente do nome do parâmetro da função.

Exemplo:

```
saudacao("João");  
// parâmetro: nome  
// argumento: "João"
```



FUNÇÕES SEM PARÂMETROS

Não recebem dados para executar suas tarefas.



```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```

saudacao()
(sem parâmetros)



Executa a ação
e realiza a tarefa.



A função não recebe dados e apenas executa a ação definida.



FUNÇÕES COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.



```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```

saudacao(nome)



nome
(parâmetro)



Usa o valor do
parâmetro na
execução.



A função recebe dados (parâmetros) e os utiliza durante a execução.



FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.



```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}
```

```
let resultado = soma(5, 3);  
console.log(resultado); // 8
```

soma(a, b)



return



valor retornado



O **return** encerra a função e devolve um valor para quem a chamou.





ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Oi!  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

Dentro da função

mensagem
= "Oi!"



Fora da função



IMPORTANTE

Após a execução,
a variável local é
descartada e não
pode ser usada
fora da função.



ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

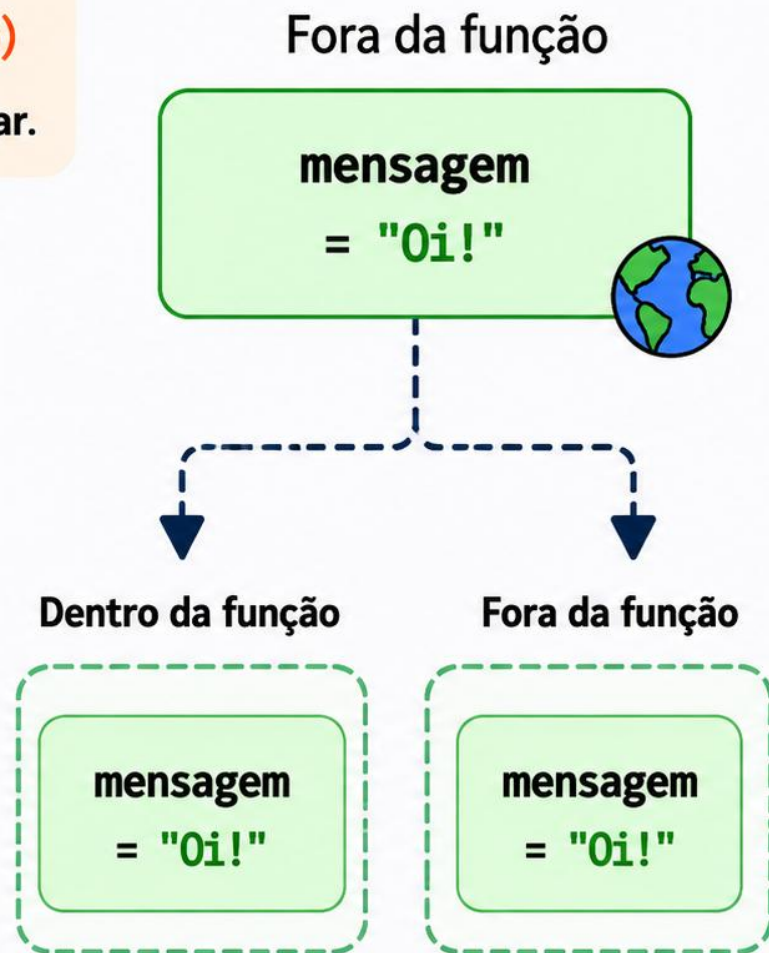
ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em **qualquer lugar**.

```
// variável global (fora de qualquer função)
let mensagem = "Oi!"; // global

function exemplo() {
  console.log(mensagem); // Oi!
}

exemplo(); // Oi!
console.log(mensagem); // Oi!
```



IMPORTANTE

Por estar no **escopo global**, a variável pode ser acessada em **todo o código**.



ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

Escopo Local (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Olá!"; // local  
  console.log(mensagem); // Ok  
}
```

```
exemplo(); // Ok  
console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem
só existe
aqui dentro

Escopo Global (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Olá!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}
```

```
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem
existe em
todo o
código



IMPORTANTE

Por estar no **escopo global**, a variável pode ser acessada em **todo o código**.



Regras de Escopo

- Tudo o que é fechado para acessos externos.
- Escopos superiores **NÃO** conseguem acessar escopos internos.
- Escopos internos **PODEM** acessar escopos superiores.

Global
(externo)

Local
(interno)



Interno → Externo
permitido.



Externo → Interno
não permitido.

PARÂMETROS E ARGUMENTOS + DICAS IMPORTANTES

Parâmetros x Argumentos



Parâmetro (definição)

É a variável declarada na função.



Argumento (na chamada)

É o valor que você passa para a função.

Exemplo



```
function multiplicar(x, y) {  
  return x * y;  
}
```

```
multiplicar(2, 4); // 8
```

Parâmetros
(x, y)

Argumentos
(2, 4)



Dicas importantes



Se houver mais de um parâmetro, forneça na ordem correta. A função pode não funcionar ou retornar valores incorretos se a ordem/tipo estiver errado.



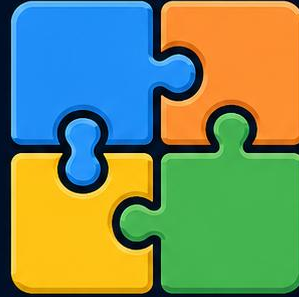
Se declarar uma variável dentro da função SEM var, let ou const, ela será **GLOBAL!**



Uma variável global criada dentro da função só existirá depois que a função for chamada ao menos uma vez!



Use sempre variáveis **LOCAIS** (let ou const).
Use **GLOBAIS** apenas quando for realmente necessário.



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.

MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

3 ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Ok  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem
só existe
aqui dentro

ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Oi!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}  
  
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem
existe em
todo o
código

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.

MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

3 ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Ok  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem só existe aqui dentro

ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Oi!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}  
  
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem existe em todo o código



REGRAS DE ESCOPO

Todo o escopo é fechado para acessos externos:

- Escopos superiores **NÃO** conseguem acessar escopos internos.
- Escopos internos **PODEM** acessar escopos superiores.

Global
(escopo externo)

Local
(escopo interno)

✓ Interno → Externo permitido

✗ Externo → Interno não permitido

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.

4 CUIDADOS COM VARIÁVEIS GLOBAIS



- Quando uma variável é declarada dentro de uma função SEM a palavra var, ela é reconhecida como uma variável GLOBAL!!!
- Uma variável global criada dentro de uma função só passará a existir depois que a função for chamada pelo menos uma vez!!!!



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

3 ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Ok  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem só existe aqui dentro

ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Oi!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}  
  
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem existe em todo o código



REGRAS DE ESCOPO

Todo o escopo é fechado para acessos externos:

- Escopos superiores **NÃO** conseguem acessar escopos internos.
- Escopos internos **PODEM** acessar escopos superiores.



✓ Interno → Externo permitido

✗ Externo → Interno não permitido

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.

4 CUIDADOS COM VARIÁVEIS GLOBAIS



- Quando uma variável é declarada dentro de uma função SEM a palavra var, ela é reconhecida como uma variável GLOBAL!!!
- Uma variável global criada dentro de uma função só passará a existir depois que a função for chamada pelo menos uma vez!!!!



5 PARÂMETROS E ARGUMENTOS

parametro1,
parametro2, ...
(definição da função)

valor1, valor2, ...
(chamada da função)

- Uma lista de parâmetros pode ser especificada na definição da função: (parametro1, parametro2, ...).
- Quando a função é chamada, os valores passados são os argumentos.
- Dentro do bloco da função, os parâmetros funcionam como variáveis locais.
- Você não consegue usar os parâmetros de uma função no script principal (fora da função).



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

3 ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Ok  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem
só existe
aqui dentro

ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Oi!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}  
  
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem
existe em
todo o
código



REGRAS DE ESCOPO

Todo o escopo é fechado para acessos externos:

- Escopos superiores **NÃO** conseguem acessar escopos internos.
- Escopos internos **PODEM** acessar escopos superiores.

Global
(escopo externo)

Local
(escopo interno)

✓ Interno → Externo permitido

✗ Externo → Interno não permitido

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.

4 CUIDADOS COM VARIÁVEIS GLOBAIS



- Quando uma variável é declarada dentro de uma função SEM a palavra var, ela é reconhecida como uma variável GLOBAL!!!
- Uma variável global criada dentro de uma função só passará a existir depois que a função for chamada pelo menos uma vez!!!!



5 PARÂMETROS E ARGUMENTOS

parametro1,
parametro2, ...
(definição da função)

valor1, valor2, ...
(chamada da função)

- Uma lista de parâmetros pode ser especificada na definição da função: (parametro1, parametro2, ...).
- Quando a função é chamada, os valores passados são os argumentos.
- Dentro do bloco da função, os parâmetros funcionam como variáveis locais.
- Você não consegue usar os parâmetros de uma função no script principal (fora da função).



6 BOAS PRÁTICAS



- O ideal é utilizar, sempre, variáveis LOCAIS, para isolar as variáveis.
- Apenas utilize variáveis GLOBAIS quando for absolutamente necessário. Isso evita a duplicação de nomes e, principalmente, o acesso e alterações indevidas dos valores das variáveis.



2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

3 ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Ok  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem
só existe
aqui dentro

ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Oi!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}  
  
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem
existe em
todo o
código



REGRAS DE ESCOPO

Todo o escopo é fechado para acessos externos:

- Escopos superiores **NÃO** conseguem acessar escopos internos.
- Escopos internos **PODEM** acessar escopos superiores.

Global
(escopo externo)

Local
(escopo interno)

✓ Interno → Externo
permitido

✗ Externo → Interno
não permitido

1 FUNÇÕES SEM E COM PARÂMETROS

SEM PARÂMETROS

Não recebem dados para executar a tarefa.

```
function saudacao() {  
  console.log("Olá!");  
}  
  
saudacao(); // Olá!
```



A função não recebe dados e apenas executa a ação definida.

COM PARÂMETROS

Recebem valores (parâmetros) para trabalhar dentro da função.

```
function saudacao(nome) {  
  console.log("Olá, " + nome + "!");  
}  
  
saudacao("Ana"); // Olá, Ana!  
saudacao("Carlos"); // Olá, Carlos!
```



A função recebe dados (parâmetros) e os utiliza durante a execução.

DICAS IMPORTANTES

- ✓ O nome da variável enviada como argumento na chamada da função não precisa ser igual ao nome do parâmetro na definição (mas precisam ser do mesmo tipo).
- ✓ Se houver mais de um parâmetro, eles precisam ser fornecidos na sequência correta. **Risco:** a função pode não funcionar ou retornar valor incorreto/esperado.

4 CUIDADOS COM VARIÁVEIS GLOBAIS



- Quando uma variável é declarada dentro de uma função SEM a palavra var, ela é reconhecida como uma variável GLOBAL!!!
- Uma variável global criada dentro de uma função só passará a existir depois que a função for chamada pelo menos uma vez!!!!



5 PARÂMETROS E ARGUMENTOS

parametro1,
parametro2, ...
(definição da função)

valor1, valor2, ...
(chamada da função)

- Uma lista de parâmetros pode ser especificada na definição da função: (parametro1, parametro2, ...).
- Quando a função é chamada, os valores passados são os argumentos.
- Dentro do bloco da função, os parâmetros funcionam como variáveis locais.
- Você não consegue usar os parâmetros de uma função no script principal (fora da função).



6 BOAS PRÁTICAS



- O ideal é utilizar, sempre, variáveis LOCAIS, para isolar as variáveis.
- Apenas utilize variáveis GLOBAIS quando for absolutamente necessário. Isso evita a duplicação de nomes e, principalmente, o acesso e alterações indevidas dos valores das variáveis.



MODULARIZAÇÃO (FUNÇÕES) EM JAVASCRIPT

Funções permitem dividir o código em blocos reutilizáveis, organizando e facilitando a manutenção.

2 FUNÇÕES COM RETORNO

Retornam um valor para ser usado fora da função.

```
function soma(a, b) {  
  return a + b; // retorna o resultado  
}  
  
let resultado = soma(5, 3);  
console.log(resultado); // 8
```



O **return** encerra a função e devolve um valor para quem a chamou.

COMO FUNCIONA

soma(5, 3)

Executa o código da função

return 8

resultado = 8

3 ESCOPO DE VARIÁVEIS

Define onde as variáveis podem ser acessadas no código.

ESCOPO LOCAL (dentro da função)

A variável só existe dentro da função.

```
function exemplo() {  
  let mensagem = "Oi!";  
  console.log(mensagem); // Ok  
}  
  
exemplo();  
// console.log(mensagem); // Erro!  
// mensagem não existe aqui fora
```

mensagem
só existe
aqui dentro

ESCOPO GLOBAL (fora da função)

A variável pode ser acessada em qualquer lugar.

```
let mensagem = "Oi!"; // global  
  
function exemplo() {  
  console.log(mensagem); // Ok  
}  
  
exemplo(); // Ok  
console.log(mensagem); // Ok
```

mensagem
existe em
todo o
código



REGRAS DE ESCOPO

Todo o escopo é fechado para acessos externos:

- Escopos superiores **NÃO** conseguem acessar escopos internos.
- Escopos internos **PODEM** acessar escopos superiores.

Global
(escopo externo)

Local
(escopo interno)

✓ Interno → Externo
permitido

✗ Externo → Interno
não permitido



RESUMO RÁPIDO



Funções
Reutilizam código e tornam o programa mais organizado.



Parâmetros
Recebem dados (entradas) para processar.



Retorno (return)
Devolve um valor da função para quem chamou.



Escopo Local
Variáveis existem apenas dentro da função.



Escopo Global
Variáveis podem ser acessadas em todo o código.



Boas Práticas
Prefira variáveis locais e use globais apenas se necessário.